

コンピュータアーキテクチャI

担当：佐々木 敬泰

第8回

高級言語からアセンブラへの変換

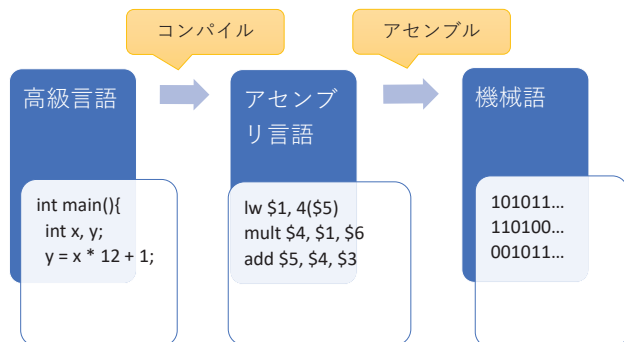
今日の内容

- 高級言語からアセンブラへの変換
 - 関数呼び出し
 - スタック
- アセンブラから機械語への変換
 - Type I
 - Type R
 - Type J

コンピュータアーキテクチャI

3

再掲 コンパイルの流れ



コンピュータアーキテクチャI

5

レジスタの役割

- \$2(\$v0)
 - 関数からの戻り値（64bitを返す場合は\$3も使う）
- \$4-\$7(\$a0-\$a3)
 - 関数への引数
- \$8-15, \$24-25(\$t0-\$t9)
 - 一時変数
- \$16-\$23(\$s0-\$s7)
 - 退避

コンピュータアーキテクチャI

7

シラバス

1. イントロダクション
2. コンピュータの構成
3. 数の表現
4. 様々なプロセッサと命令セット
5. 基本命令
6. メモリアクセス命令
7. 分岐命令
8. 高級言語からアセンブラへの変換
9. プログラムの翻訳と起動
10. 加減算
11. 乗算アルゴリズムと乗算器
12. 除算アルゴリズムと除算器
13. 浮動小数点
14. 浮動小数点の乗算と誤差
15. 定期試験

コンピュータアーキテクチャI

2

高級言語からアセンブラへの変換

コンピュータアーキテクチャI

4

再掲 MIPSのレジスタ割り当て

• レジスタの用途

名前	レジスタ番号	用途	呼び出し時に退避
\$zero	0	常にゼロ	不要
\$v0-\$v1	2-3	結果、式の評価	不要
\$a0-\$a3	4-7	関数の引数	不要
\$t0-\$t7	8-15	一時的な変数	不要
\$s0-\$s7	16-23	退避	必要
\$t8-\$t9	24-25	予備の一時変数	不要
\$gp	28	グローバルポインタ	必要
\$sp	29	スタックポインタ	必要
\$fp	30	フレームポインタ	必要
\$ra	31	戻りアドレス	必要

\$26-\$27はオペレーティングシステム用に予約されている

6

おさらい

```
int sum(int num){
  int i, ans = 0;
  for(i = 0; i < num; i = i + 1){
    ans = ans + i;
  }
  return ans;
}
```

sum:

```
add $2, $0, 0 # ans
add $8, $0, $0 # i
L1: slt $9, $8, $4
    beq $9, $2, L2
    add $2, $2, $8
    addi $8, $8, 1
    j L1
L2: jr $31
```

コンピュータアーキテクチャI

8

関数内の作業レジスタ

• $\$8 = 1 + \text{sum}(3, 4);$

```
addi    $8, $0, 1    # $8 = 1
addi    $4, $0, 3    # 引数1
addi    $5, $0, 4    # 引数2
jal      sum          # $2 = sum(3, 4);
addi    $8, $8, $2    # $8 = $8 + $2
```

【解決策1】

\$8の代わりに、関数呼び出ししても値が変わらないことが保証されている\$16(\$s0)を使う

【解決策2】

```
addi$29, $29, -4
sw    $8, 0($29)
jal    sum
lw    $8, 0($29)
addi$29, $29, 4
```

として、sum関数を呼ぶ前に\$8を自分で退避する

スピルアウト

• C言語の変数はレジスタに割り当てられる

```
int func(int a, int b){
    int x, y, z;

    x = a + b;
    y = func(b) + x;
    :
    :
```

変数	レジスタ
a	\$4
b	\$5
x	\$8
y	\$9
z	\$10

• レジスタ本数は有限

- レジスタ本数が足りない場合は？
- 配列の場合は？

➡ スタックを使う

ローカル変数とスタック

• 関数の引数や戻りアドレス、退避が必要なレジスタ(\$s0-\$s7や\$raなど)に加え、ローカル変数もスタック上に確保する

しかし

• スタックポインタはスタックへのアクセスで移動する
⇒「フレームポインタ(\$fp)」を用いる

ローカル変数	← \$sp
:	:
ローカル変数	:
退避レジスタ	:
:	:
退避レジスタ	:
戻りアドレス	:
引数	:
:	:
引数	← \$fp

C言語の例

```
int func(int a, int b, int c, int d, int e, int f){
    int x, y;

    x = e;
    y = f;

    return x+y;
}
```

対応するアセンブラ

func:		\$fp相対アドレス	データ
addiu	\$sp, \$sp, -40	20	(未使用)
sw	\$ra, 36(\$sp)	24	y(=f)
sw	\$fp, 32(\$sp)	28	x(=e)
add	\$fp, \$0, \$sp	32	\$30(\$fp)
lw	\$2, 56(\$fp)	36	\$31(\$ra)
sw	\$2, 28(\$fp)	40	a
lw	\$2, 60(\$fp)	44	b
sw	\$2, 24(\$fp)	48	c
lw	\$3, 28(\$fp)	52	d
addu	\$2, \$3, \$2	56	e
add	\$sp, \$0, \$fp	60	f
lw	\$ra, 36(\$sp)		
lw	\$fp, 32(\$sp)		
addiu	\$sp, \$sp, 40		
j	\$ra		

対応するアセンブラ

func:		\$fp相対アドレス	データ
addiu	\$sp, \$sp, -40	20	(未使用)
sw	\$ra, 36(\$sp)	24	y(=f)
sw	\$fp, 32(\$sp)	28	x(=e)
add	\$fp, \$0, \$sp	32	\$30(\$fp)
lw	\$2, 56(\$fp)	36	\$31(\$ra)
sw	\$2, 28(\$fp)	40	a
lw	\$2, 60(\$fp)	44	b
sw	\$2, 24(\$fp)	48	c
lw	\$3, 28(\$fp)	52	d
addu	\$2, \$3, \$2	56	e
add	\$sp, \$0, \$fp	60	f
lw	\$ra, 36(\$sp)		
lw	\$fp, 32(\$sp)		
addiu	\$sp, \$sp, 40		
j	\$ra		

【前処理】

- スタックを40バイト確保
- \$raと\$fpをスタック上に保存
- \$fp = \$sp

対応するアセンブラ

func:		\$fp相対アドレス	データ
addiu	\$sp, \$sp, -40	20	(未使用)
sw	\$ra, 36(\$sp)	24	y(=f)
sw	\$fp, 32(\$sp)	28	x(=e)
add	\$fp, \$0, \$sp	32	\$30(\$fp)
lw	\$2, 56(\$fp)	36	\$31(\$ra)
sw	\$2, 28(\$fp)	40	a
lw	\$2, 60(\$fp)	44	b
sw	\$2, 24(\$fp)	48	c
lw	\$3, 28(\$fp)	52	d
addu	\$2, \$3, \$2	56	e
add	\$sp, \$0, \$fp	60	f
lw	\$ra, 36(\$sp)		
lw	\$fp, 32(\$sp)		
addiu	\$sp, \$sp, 40		
j	\$ra		

- \$2=e
- x=\$2
- \$2=f
- y=\$2

対応するアセンブラ

func:		\$fp相対アドレス	データ
addiu	\$sp, \$sp, -40	20	(未使用)
sw	\$ra, 36(\$sp)	24	y(=f)
sw	\$fp, 32(\$sp)	28	x(=e)
add	\$fp, \$0, \$sp	32	\$30(\$fp)
lw	\$2, 56(\$fp)	36	\$31(\$ra)
sw	\$2, 28(\$fp)	40	a
lw	\$2, 60(\$fp)	44	b
sw	\$2, 24(\$fp)	48	c
lw	\$3, 28(\$fp)	52	d
addu	\$2, \$3, \$2	56	e
add	\$sp, \$0, \$fp	60	f
lw	\$ra, 36(\$sp)		
lw	\$fp, 32(\$sp)		
addiu	\$sp, \$sp, 40		
j	\$ra		

- \$3=e
- \$2=f
- \$2=\$3+\$2

対応するアセンブラ

func:

```

addiu    $sp, $sp, -40
sw       $ra, 36($sp)

```

【後処理】

- $\$sp = \fp
- $\$ra$ と $\$fp$ をスタックから読み出して元に戻す
- $\$sp$ を戻す
- 呼び出し元にジャンプ

```

lw       $ra, 36($sp)
addiu    $sp, $sp, 40
j        $ra

```

\$fp相対アドレス	データ
20	(未使用)
24	y(=f)
28	x(=e)
32	\$30(\$fp)
36	\$31(\$ra)
40	a
44	b
48	c
52	d
56	e
60	f

グローバルポインタ

- ローカル変数は、スタック上にメモリを確保し、 $\$fp$ を使ってアクセス
- グローバル変数は？
 - グローバルポインタ($\$gp$)を使う
- 例)
 - $lw \$8, 120(\$gp)$
- $\$gp$ はグローバル変数のある領域の真ん中を指す
 - lw の即値は符号付き
 - ⇒最大65536B(2^{16})のデータにアクセス可



アセンブラから機械語への変換

命令（機械語）

- コンピュータ内での命令の表現
- MIPSでは、命令は固定長(32bit)の数値として表現する。
- 32bitを決められたビット数で区切る
 - 区切られたビット列をフィールドと呼ぶ
- フィールドの意味や役割は命令によって変わる

MIPSの命令タイプ

- MIPSの命令フォーマット
 - Type I
 - 即値（定数）を取る書式
 - $ADDI \$4, \$6, 0x1234 \rightarrow \$4 = \$6 + 0x1234$
 - Type J
 - 分岐（ジャンプ）命令用の特殊フォーマット
 - $J \text{ addr} \rightarrow PC = \{PC[31:28], \text{addr}[25:0], 2'b00\} = (PC \& 0xfc) | (\text{addr} \ll 2)$
 - Type R
 - オペランドとしてレジスタのみを取る
 - $ADD \$4, \$5, \$6 \rightarrow \$4 = \$5 + \6

Type I 命令

ADDI
レジスタ = レジスタ + 即値（定数）

機械語(Type I)

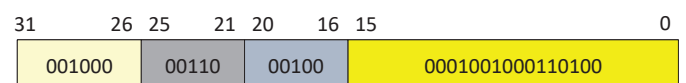
- オペコード（命令）、ソースレジスタ、ターゲットレジスタ、即値を32ビットに詰込む



- ADDI \$4, \$6, 0x1234
 - ADDIのオペコードは001000b
 - ターゲットは\$4 = 00100b
 - ソースは\$6 = 00110b
 - 即値は0x1234 = 0001001000110100b

機械語(Type I)

- オペコード（命令）、ソースレジスタ、ターゲットレジスタ、即値を32ビットに詰込む



- ADDI \$4, \$6, 0x1234
 - ADDIのオペコードは001000b
 - ターゲットは\$4 = 00100b
 - ソースは\$6 = 00110b
 - 即値は0x1234 = 0001001000110100b

0x20c41234

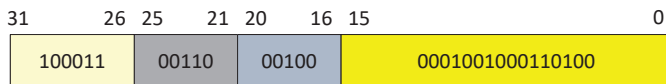
機械語

Type I 命令

LW
 ADR = レジスタ + 即値 (定数)
 レジスタ = メモリ[ADR]

機械語(Type I)

- オペコード (命令)、ソースレジスタ、ターゲットレジスタ、即値を32ビットに詰込む



- LW \$4, 0x1234(\$6)

- LW のオペコードは 100011b
- ターゲットは \$4 = 00100b
- ソースは \$6 = 00110b
- 即値は 0x1234 = 0001001000110100b

0x8cc41234

機械語

35

機械語(Type I)

- オペコード (命令)、ソースレジスタ、ターゲットレジスタ、即値を32ビットに詰込む



- LW \$4, 0x1234(\$6)

- LW のオペコードは 100011b
- ターゲットは \$4 = 00100b
- ソースは \$6 = 00110b
- 即値は 0x1234 = 0001001000110100b

34

Type J 命令

J
 PC = 指定アドレス

機械語(Type J)

- オペコード (命令) と即値を32ビットに詰込む



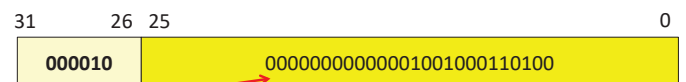
- J 0x1234

- J のオペコードは 000010b
- 即値は 0x1234 = 0000000000000001001000110100b

37

機械語(Type J)

- オペコード (命令) と即値を32ビットに詰込む



- J 0x1234

- J のオペコードは 000010b
- 即値は 0x1234 = 0000000000000001001000110100b

0x08001234

機械語

38

Type R 命令

ADD
 レジスタ = レジスタ + レジスタ

機械語(Type R)

- オペコード (命令)、ソースレジスタ、ターゲットレジスタ、ディスティネーションレジスタ (書き込み先)、即値を32ビットに詰込む



- ADD \$4, \$5, \$6

- ADD のオペコードは 000000b
- 機能は 100000b
- 書き込み先は \$4 = 00100b
- ソースは \$5 = 00101b
- ターゲットは \$6 = 00110b

~重要~

- Type I
結果はターゲットレジスタ
- Type R
結果は書き込み先レジスタ

40

機械語(Type R)

- オペコード（命令）、ソースレジスタ、ターゲットレジスタ、ディスティネーションレジスタ（書込み先）、即値を32ビットに詰込む

31	26	25	21	20	16	15	11	10	6	5	0
000000	ソース	ターゲット	書込み先	シフト量	機能						

Type Rはここが全部ゼロ！

ここはシフト命令などで使用（今回は無視してよい）

本当の命令(機能)はここに入ってる

実行手順

1. [31:26]が000000(Type R)かどうか確かめる
2. もし、Type Rなら[5:0]を見て命令を解釈

41

演習

1. 次のプログラムを機械語に直せ
ORI \$1, \$0, 0x1234
SUB \$2, \$0, \$1
2. 次の機械語をアセンブラ言語に戻せ
0x8c230029
0x00431826
※ この操作を逆アセンブルと呼ぶ

42

演習の解答1

1. 次のプログラムを機械語に直せ

43

演習の解答2

2. 次の機械語をアセンブラ言語に戻せ

44